**Team Members**
**Ernest udoezika (e2601991)**

**Tadvin Masud (e2401843)**

**Saharsh Verma (e2401815)**

**Define and Analyse the Project Requirements**

The project is a multi-hotel room reservation web application. It will allow customers to search for hotels and available rooms, make and manage reservations, and complete a simulated payment. Hotel staff will manage rooms, reservations and guests, while administrators will manage hotels and users. The application will be developed using Java and Spring Boot.

**1. Identify the Stakeholders and User Needs**

| Stakeholder | Responsibilities | Main Needs |
|---|---|---|
| **Customer/Guest** | Search and book rooms | Easy hotel search, accurate availability, simple booking, secure account, reservation management and simulated payment |
| **Hotel Staff/Receptionist** | Manage hotel operations | Manage rooms, availability, reservations, customers, check-in and check-out |
| **Hotel Manager** | Oversee hotel operations | Accurate room and reservation information and effective room management |
| **System Administrator** | Manage the booking platform | Manage hotels, users, staff accounts and system access |
| **Platform Owner** | Overall system/business responsibility | Secure, reliable, maintainable and scalable application |
| **Development Team** | Develop and maintain the application | Clear requirements, modular architecture, maintainable code and reliable technologies |

## 2. Define the Overall Application Requirements

### Functional Requirements

The system shall:

1. Allow customers to register and log in.
2. Allow customers to search for hotels by location and dates.
3. Display available rooms based on selected dates and number of guests.
4. Display hotel and room information, including room type, capacity and price.
5. Allow customers to make reservations.
6. Allow customers to modify and cancel reservations.
7. Allow customers to view their booking history.
8. Provide a simulated payment process.
9. Allow hotel staff to add, edit and remove rooms.
10. Allow hotel staff to manage room availability and prices.
11. Allow hotel staff to create, modify and cancel reservations.
12. Allow hotel staff to manage customer information.
13. Allow hotel staff to check guests in and out.
14. Allow administrators to manage hotels.
15. Allow administrators to manage users and staff accounts.
16. Use role-based access control to restrict functions.
17. Prevent double booking of rooms.

### Performance Requirements

- Normal operations should respond within approximately 2 seconds under expected system load.
- Hotel and room searches should return results efficiently.
- The system should support multiple users at the same time.
- Database queries should be optimised for frequently used searches.

### Usability Requirements

- The interface should be simple and intuitive.
- Customers should be able to complete a booking without extensive training.
- Navigation should be consistent.
- Forms should clearly indicate required fields.
- Error messages should be understandable.
- The booking process should use as few unnecessary steps as possible.

### Reliability and Availability Requirements

- The system must prevent double bookings.
- Reservation data must remain consistent.

- Failed transactions should not create incomplete bookings.
- Database transactions should be used for important reservation operations.
- The application should handle failures without exposing technical information to users.
- The system should recover gracefully from temporary failures.

**Security Requirements**

The system must protect:

- Customer personal information.
- User accounts and passwords.
- Reservation information.
- Payment information used by the simulated payment system.
- Hotel and staff information.
- Administrator functions.

Security requirements include:

- Passwords must be securely hashed.
- Users must authenticate before accessing protected functionality.
- Role-based authorisation must be implemented.
- Customers must only access their own reservations.
- Administrative functions must be restricted.
- User input must be validated.
- HTTPS should be used in production.

**Maintainability Requirements**

- The application should use a modular architecture.
- Business logic should be separated from the user interface and database access.
- Java classes and methods should have clear names.
- Code duplication should be minimised.
- The system should be documented.
- Automated tests should be created for important functionality.
- The architecture should allow future features to be added without major changes.

**Compatibility and Integration Requirements**

The application will use:

- **Java**
- **Spring Boot**

- **Spring Data JPA/Hibernate**
- **PostgreSQL**
- **HTML/CSS/JavaScript**
- **Maven**
- **Git**

**Constraints**

- The main backend technology must be Java/Spring Boot.
- The project is a coursework project, so the scope must remain manageable.
- Real online payment is outside the current scope.
- Email and SMS notifications are outside the scope.
- Promotions and discounts are not required.
- Report generation is not required.
- The system requires persistent database storage.
- Personal data must be handled securely and according to applicable data-protection requirements.

## 3. Partition the Application into Logical Subsystems

### 3.1 User and Authentication Subsystem

**Purpose:** Manage users, login and access control.

**Responsibilities:**

- Customer registration
- Login/logout
- Password management
- User roles
- Authentication and authorisation

**Inputs:**

- Name
- Email
- Password
- Login credentials
- User role

**Outputs:**

- Authentication result
- User account information
- Access permissions

**Requirements:**

- Secure authentication
- Password hashing
- Role-based access control
- Protection against unauthorised access

### 3.3 Room and Availability Subsystem

**Purpose:** Manage rooms and determine their availability.

**Responsibilities:**

- Add/edit/remove rooms
- Manage room types
- Manage room prices
- Check availability
- Manage room status

**Inputs:**

- Hotel ID
- Room ID
- Room type
- Price
- Capacity
- Check-in/check-out dates

**Outputs:**

- Available rooms
- Room information
- Room status

**Requirements:**

- Availability must be accurate.
- Double booking must be prevented.
- Room information must be consistent.
- Availability searches should be fast.

### 3.4 Search Subsystem

**Purpose:** Help customers find suitable rooms.

**Inputs:**

- Location
- Check-in date
- Check-out date
- Number of guests
- Room type
- Other search filters

**Outputs:**

- Available rooms
- Prices
- Room information

**Requirements:**

- Search results should be returned quickly.
- Unavailable rooms should not be displayed as available.
- Search should be easy to use.

### 3.5 Reservation Subsystem

**Purpose:** Manage the complete reservation lifecycle.

**Responsibilities:**

- Create reservations
- Modify reservations
- Cancel reservations
- View reservations
- Manage reservation status

**Inputs:**

- Customer ID
- Room ID
- Dates
- Number of guests
- Payment status

**Outputs:**

- Reservation ID
- Reservation details
- Reservation status
- Booking history

**Requirements:**

- Reservations must be stored reliably.
- Availability must be checked before confirmation.
- Double booking must be prevented.
- Only authorised users can modify reservations.

### 3.6 Payment Subsystem

**Purpose:** Process simulated payments.

**Inputs:**

- Reservation ID
- Payment amount
- Mock payment details

**Outputs:**

- Payment success/failure
- Payment status
- Transaction ID

**Requirements:**

- Payment results must be stored.
- A successful payment should allow the reservation to become confirmed.
- Failed payments must not result in an incorrectly confirmed reservation.

### 3.7 Check-in/Check-out Subsystem

**Purpose:** Manage guest arrival and departure.

**Inputs:**

- Reservation ID
- Customer information
- Room information

**Outputs:**

- Updated reservation status
- Updated room status

**Requirements:**

- Only authorised hotel staff can perform check-in/check-out.
- A cancelled reservation cannot be checked in.
- Room status must be updated correctly.

### 3.8 Administration Subsystem

**Purpose:** Provide system-wide management.

**Responsibilities:**

- Manage users
- Manage hotel staff
- Manage roles and permissions

**Inputs:**

- User information
- Staff information
- Role information

**Outputs:**

- Updated user records
- Access permissions

**Requirements:**

- Only administrators can access administrative functions.
- Changes must be validated.
- Administrative operations must be secure.

## 4. Quality Function Deployment (QFD)

QFD translates customer needs into technical requirements.

**Customer/User Needs**

| Customer Need | Type | Importance |
|---|---|---|
| Easy room search | Normal | 5 |
| Easy reservation process | Normal | 5 |
| Accurate room availability | Expected | 5 |
| Secure personal information | Expected | 5 |
| Reliable booking | Expected | 5 |
| Easy-to-use interface | Expected | 5 |
| Fast search | Expected | 4 |
| Modify reservation | Normal | 4 |
| Cancel reservation | Normal | 4 |
| View booking history | Normal | 4 |
| Simulated payment | Normal | 4 |
| Clear room/hotel information | Normal | 4 |
| Additional search filters | Fun | 3 |

**Technical Requirements**

The customer needs are translated into the following technical requirements:

| ID | Technical Requirement |
|---|---|
| TR1 | Room availability validation |
| TR2 | Double-booking prevention |
| TR3 | Reservation transaction management |
| TR4 | Database integrity |
| TR5 | Search response performance |
| TR6 | Authentication |
| TR7 | Role-based authorisation |

| ID | Technical Requirement |
|---|---|
| TR8 | Password hashing |
| TR9 | Input validation |
| TR10 | User-friendly interface |
| TR11 | Reservation modification/cancellation |
| TR12 | Booking history storage |
| TR13 | Mock payment processing |
| TR14 | Error handling |
| TR15 | Modular Spring Boot architecture |

**QFD Relationship**

Relationship values:

- **9 = Strong**
- **3 = Medium**
- **1 = Weak**
- **0 = No significant relationship**

| Customer Need | Importance | Availability | Reservation | Security | Performance | UI | Data Integrity | Payment |
|---|---|---|---|---|---|---|---|---|
| Easy room search | 5 | 9 | 3 | 0 | 9 | 9 | 3 | 0 |
| Easy reservation | 5 | 3 | 9 | 3 | 3 | 9 | 9 | 3 |
| Accurate availability | 5 | 9 | 9 | 0 | 3 | 1 | 9 | 0 |
| Secure information | 5 | 0 | 3 | 9 | 0 | 3 | 9 | 3 |
| Reliable booking | 5 | 9 | 9 | 3 | 3 | 1 | 9 | 3 |
| Easy interface | 5 | 1 | 3 | 1 | 3 | 9 | 1 | 3 |
| Fast search | 4 | 3 | 1 | 0 | 9 | 3 | 3 | 0 |
| Modify reservation | 4 | 3 | 9 | 3 | 1 | 3 | 9 | 1 |

| Customer Need | Importance | Availability | Reservation | Security | Performance | UI | Data Integrity | Payment |
|---|---|---|---|---|---|---|---|---|
| Cancel reservation | 4 | 3 | 9 | 3 | 1 | 3 | 9 | 0 |
| Booking history | 4 | 0 | 3 | 3 | 3 | 3 | 9 | 1 |
| Simulated payment | 4 | 0 | 3 | 3 | 0 | 3 | 9 | 9 |

**QFD Priorities**

The QFD analysis shows that the technical requirements that deserve the most attention are:

1. **Database integrity and consistency**
2. **Room availability validation**
3. **Double-booking prevention**
4. **Reservation transaction processing**
5. **Security and access control**
6. **Search performance**
7. **Usability of the booking interface**
8. **Reliable simulated payment processing**

These priorities are based on the importance assigned to customer needs and the strength of their relationship with the technical requirements.

**Requirement Classification**

**Normal requirements:**

- Room search
- Reservation
- Reservation modification
- Reservation cancellation
- Booking history
- Simulated payment
- Room management

**Expected requirements:**

- Security
- Accurate availability

- Reliable booking
- No double booking
- Good performance
- Easy navigation
- Data integrity
- Clear error handling